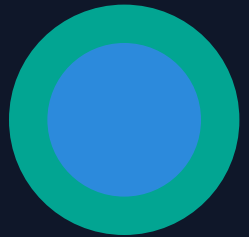


Deep Learning Hyperparameter Optimization

for IPL Match Winner Prediction

Jagdeep | AI & Applied ML | Fanshawe College

Deep Learning Optimization Project | April 2026



Problem Statement & Motivation



Why IPL Prediction?

High-profile T20 league with complex match dynamics. Outcome depends on team form, venue, toss, and seasonal patterns — a challenging tabular classification task.



Why Optimization?

Neural networks are sensitive to hyperparameters. Default settings often leave performance on the table. Systematic tuning can shift the precision-recall tradeoff significantly.



Why Compare Methods?

Random Search samples broadly. Bayesian Optimization (Optuna) learns from prior trials. Comparing them reveals whether guided search outperforms brute-force exploration on this problem.

Dataset & Prediction Setup



Data Overview

- Source: (Kaggle) IPL match-level dataset spanning seasons(2008-2024)
- 328 matches retained as the working sample after filtering incomplete records and building historical features
- Binary target: predict match winner (team1 win = 1)
- Pre-match features only — no in-match or post-match columns used
- Features: season year, month, toss winner/decision, city/venue, team prior win rates, head-to-head records
- Post-match columns (winner, MOTM, result margin) excluded from feature set to avoid data leakage

328

Working Sample

51.8%

Majority Class

Pre-Match Features Only

Seed 42

Reproducibility

Preprocessing & Feature Engineering

1

Clean

Handle missing values, remove post-match columns, standardize franchise names across seasons

2

Encode

One-hot encode categoricals (teams, city, venue, toss decision). Numeric features kept as-is

3

Scale

StandardScaler on numeric features for stable gradient descent and fair regularization

4

Split

Chronological split: oldest 70% train, next 15% validation, newest 15% test — simulates real forecasting

Why preprocessing matters for optimization:

Proper scaling ensures gradient-based optimizers converge reliably across different learning rates and architectures. Without it, hyperparameter search results become noisy and unreproducible.

Baseline Model & Architecture



Keras MLP Architecture

Input → Dense(64, ReLU) → Dropout(0.2)
→ Dense(32, ReLU) → Dropout(0.2)
→ Dense(1, Sigmoid)

Loss: Binary Cross-Entropy

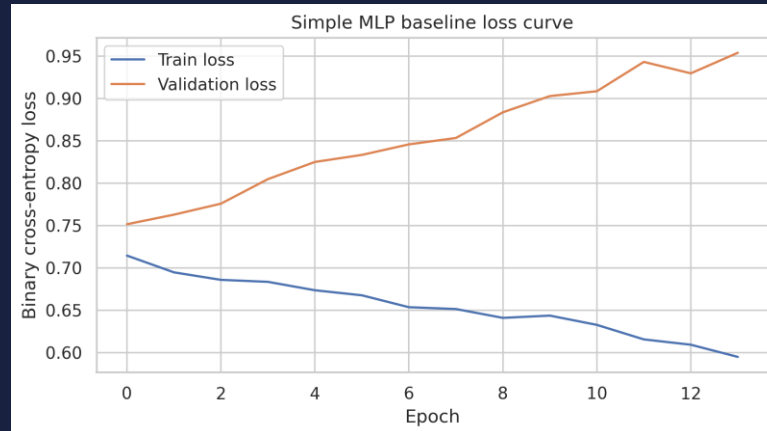
Optimizer: Adam (lr = 1e-3)

Epochs: up to 60 with Early Stopping

Early stop monitor: Validation AUC

Evaluated on: Acc, F1, ROC-AUC, Precision, Recall,

Log Loss



Baseline training / validation loss curve

Why MLP for tabular data?

MLPs are a standard deep learning baseline for tabular classification. They are fast to train, easy to tune, and serve as a fair substrate for comparing optimization methods.

Hyperparameter Optimization Setup

Hyperparameter	Search Range	Type
Learning Rate	1e-4 — 3e-2	Log-uniform
Batch Size	{16, 32, 64, 128}	Categorical
Dropout Rate	{0.0, 0.1, 0.2, 0.3, 0.4, 0.5}	Categorical
Hidden Units	{32, 64, 128, 256}	Categorical
Optimizer	{adam, rmsprop, sgd}	Categorical

Fixed seed = 42 for model reproducibility. Both methods share the identical search space and trial budget.

Random Search

50 trials

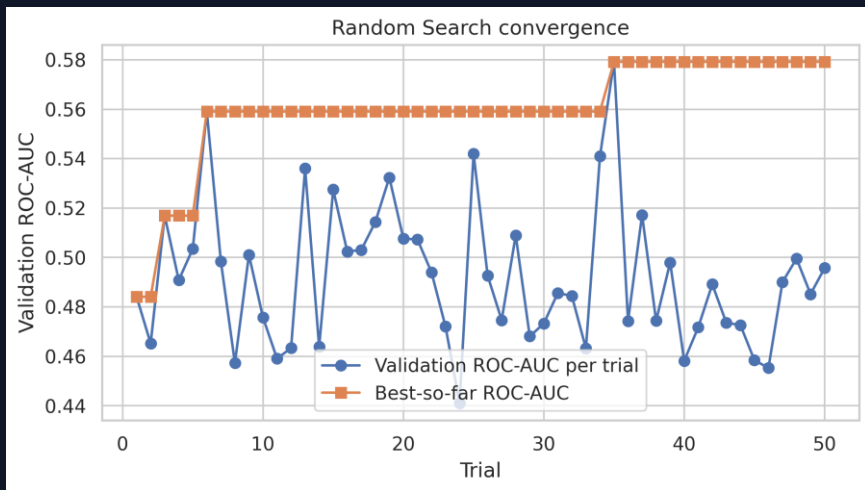
Samples hyperparameters uniformly at random from the search space. Simple, embarrassingly parallel, provides broad coverage.

Bayesian Optimization

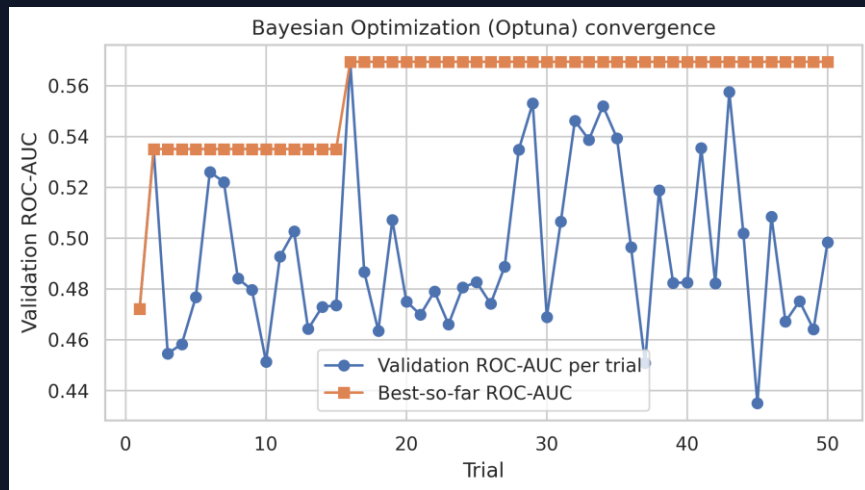
50 trials (Optuna TPE)

Uses a Tree-structured Parzen Estimator to model promising regions. Each trial informs the next — guided, sequential search.

Random Search vs Bayesian Optimization



Random Search — validation ROC-AUC convergence



Optuna (Bayesian) — validation ROC-AUC convergence

	Random Search	Bayesian (Optuna)
Trials	50	50
Runtime	173.2 s	188.0 s
Best Val Accuracy	0.6012	0.5644
Best Val ROC-AUC	0.5792	0.5694

Best Hyperparameters Found

Hyperparameter	Random Search	Bayesian (Optuna)
Learning Rate	4.56×10^{-4}	1.97×10^{-4}
Batch Size	128	128
Dropout Rate	0.50	0.20
Hidden Units	64	256
Optimizer	RMSProp	SGD

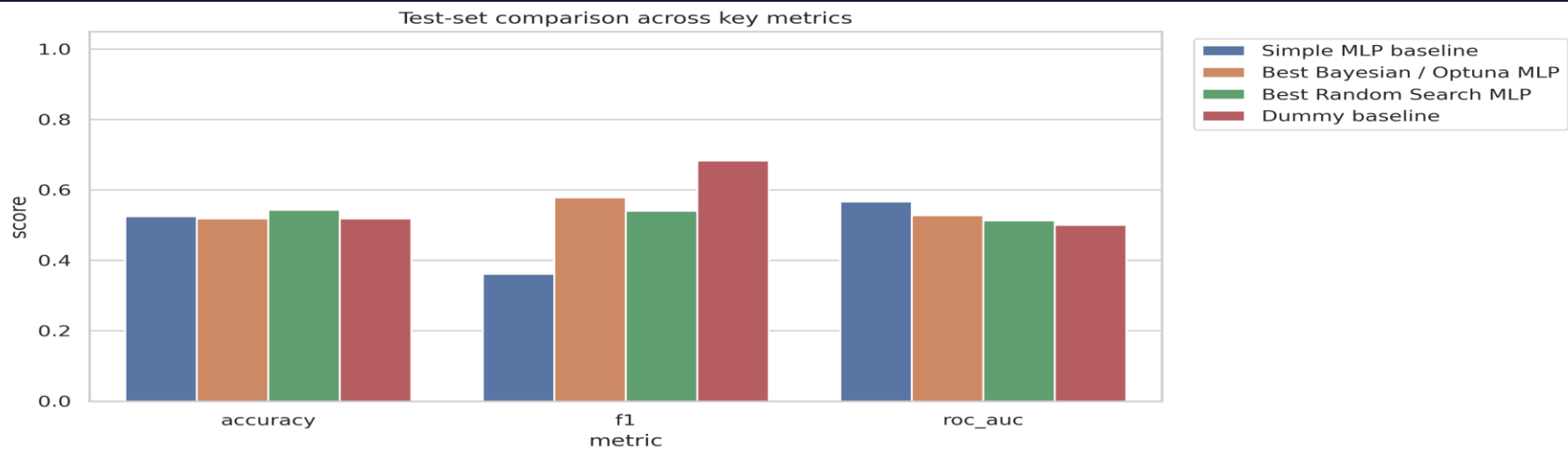
Random Search Strategy

Higher LR + heavier dropout (0.5) + smaller network (64 units). A lean, regularized model with RMSProp — favors conservative predictions.

Optuna Strategy

Lower LR + lighter dropout (0.2) + larger network (256 units). A capacity-heavy model with SGD — predicts more aggressively, higher recall.

Test Set Results



Test set performance across four models and six metrics

Key Findings

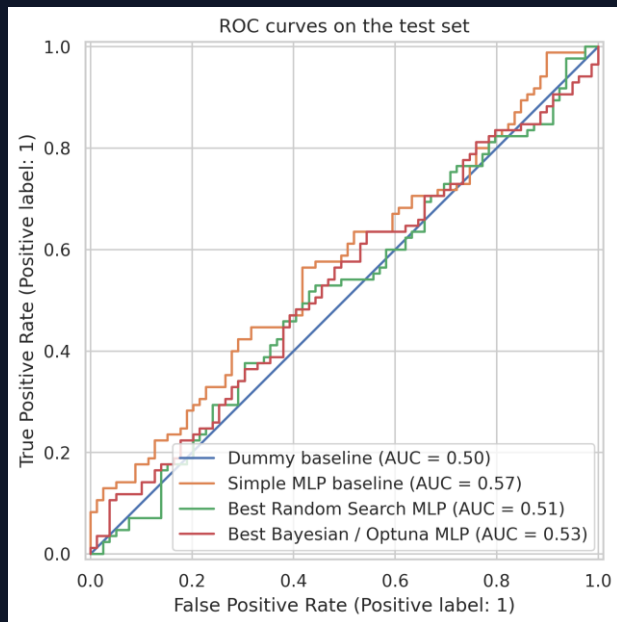
Best Accuracy (54.3%): Random Search MLP

Best ROC-AUC (0.566) & Log Loss (0.689): Simple MLP Baseline — best ranking and calibration

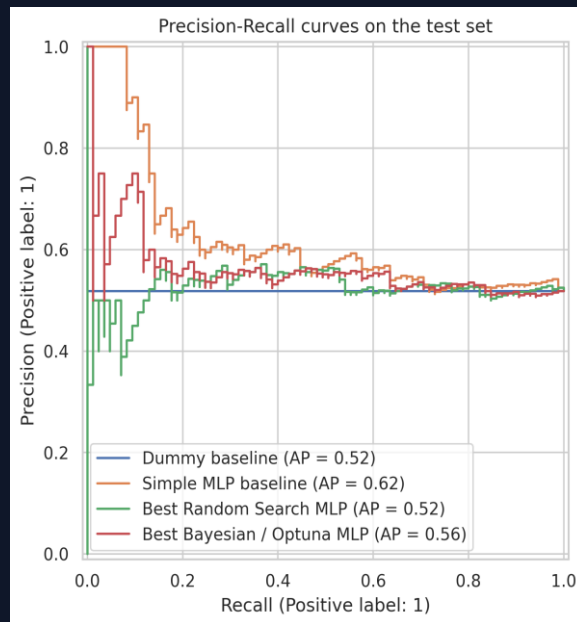
Best F1 (0.578) & Recall (63.5%): Optuna MLP — most aggressive positive-class detection among tuned models

Takeaway: Optimization shifted the precision-recall tradeoff rather than producing a universally superior model

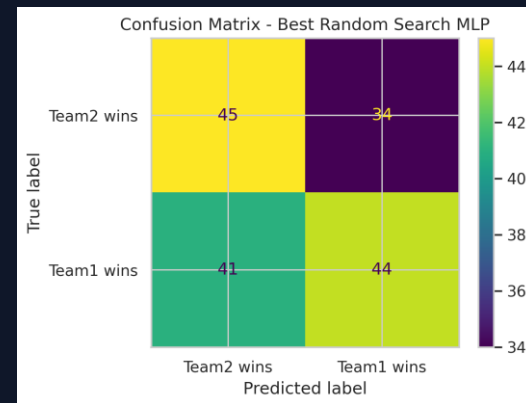
Evaluation Visuals



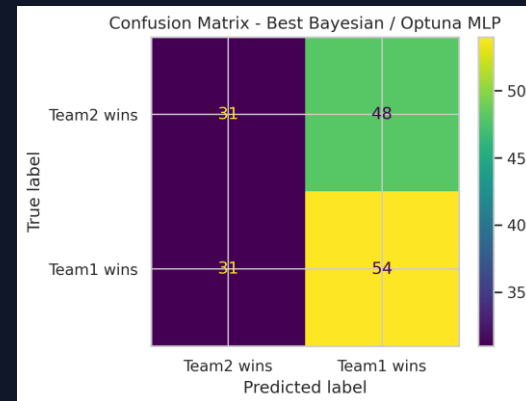
ROC Curves — ranking performance



Precision-Recall Curves



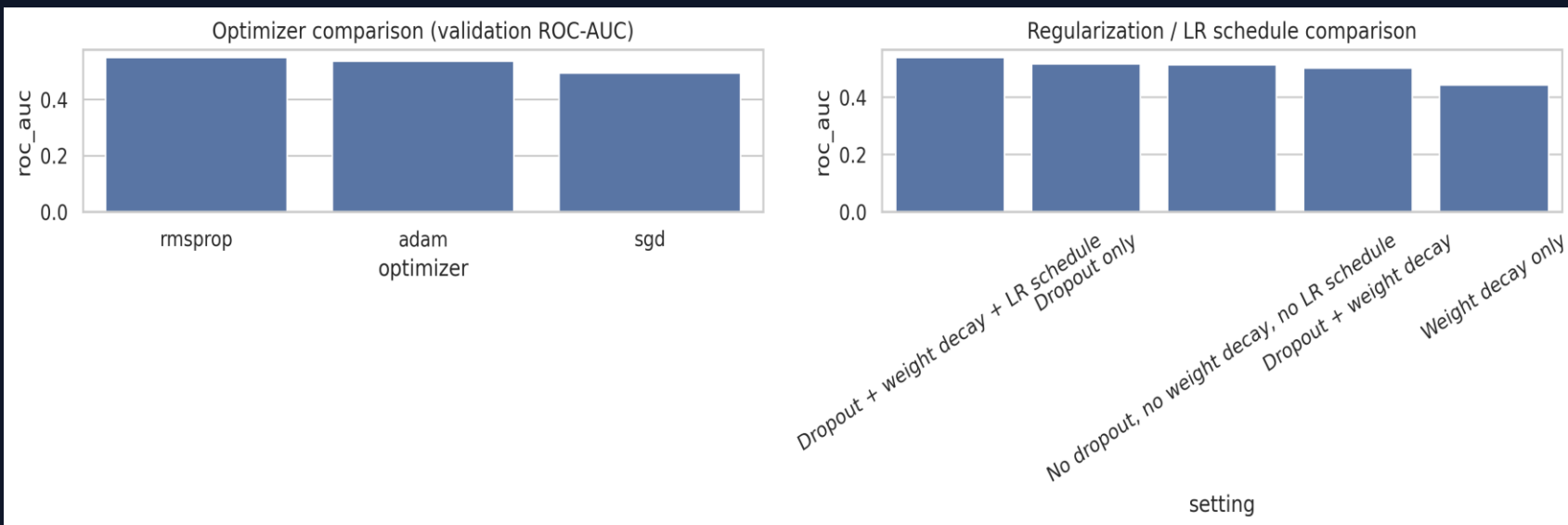
Random Search CM



Optuna CM

Optuna MLP predicts positives more aggressively (higher recall). Random Search MLP is more balanced but misses more positives. Baseline shows strongest ROC separation despite lower recall.

Optimizer & Regularization



Optimizer comparison (left) and regularization (right) on validation set

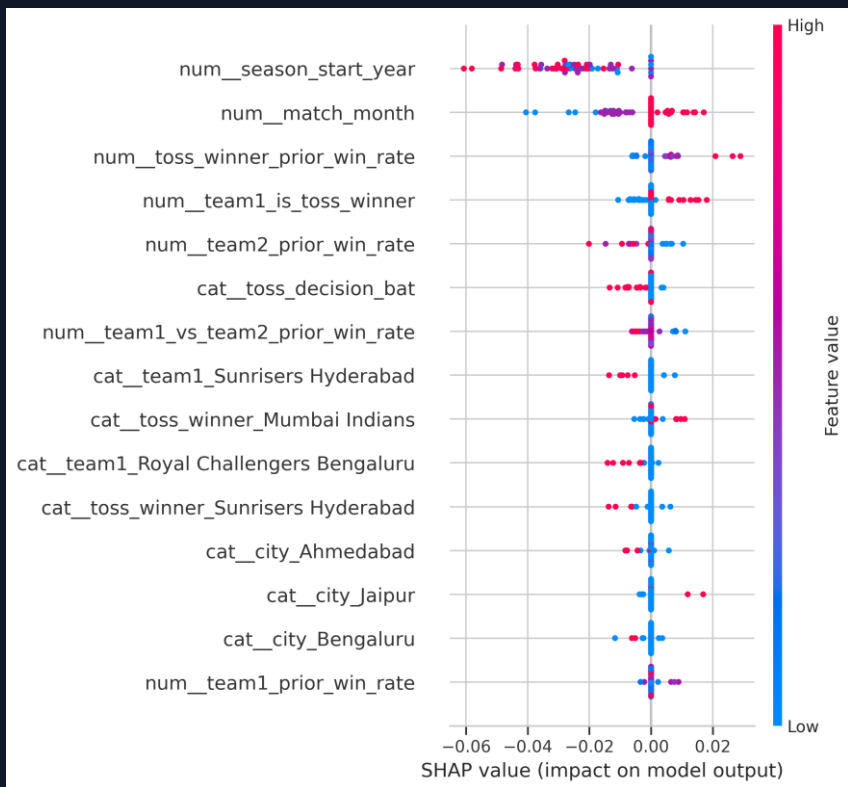
Optimizer Comparison

Adam achieved the best validation accuracy (0.564). SGD had the highest recall (0.653) but lowest accuracy (0.485). RMSProp sat between the two. Optimizer choice strongly affects the precision-recall tradeoff.

Regularization

Without any regularization, recall spikes to 91.7% but accuracy drops to 47.2%. Dropout alone restores balance. Adding weight decay and LR schedule further shifts behavior. Regularization acts as a precision-recall lever.

Model Explainability (SHAP)



Global SHAP Feature Importance

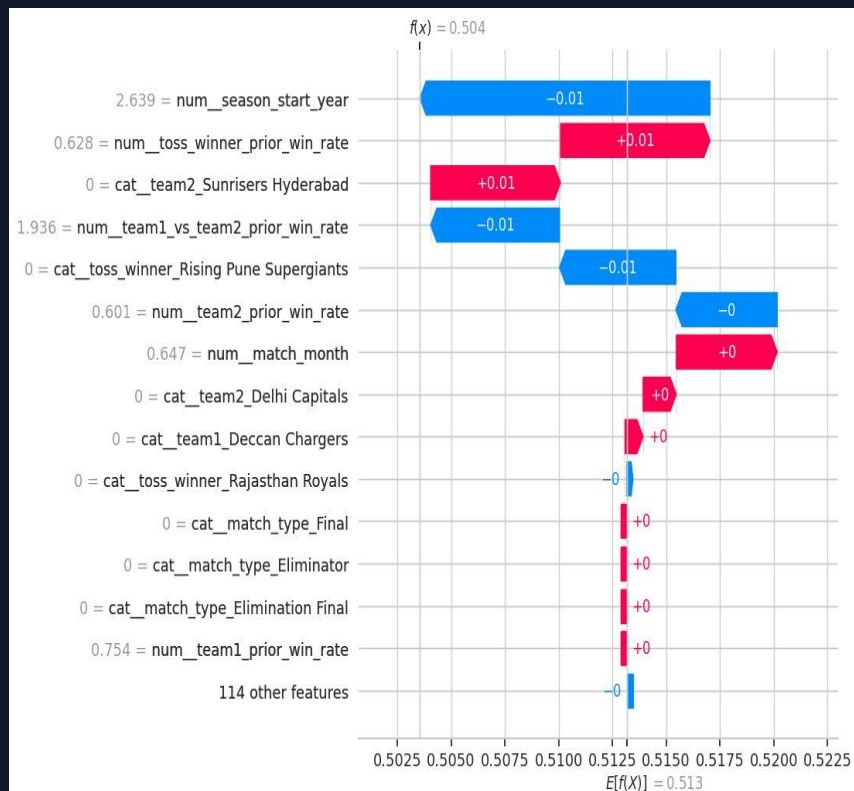
Rank	Feature	Mean SHAP
1	season_start_year	0.0248
2	match_month	0.0090
3	toss_winner_prior_win_rate	0.0031
4	team1_is_toss_winner	0.0030
5	team2_prior_win_rate	0.0019
6	toss_decision_bat	0.0016

Top features by mean |SHAP| value

Key Insights

- season_start_year dominates — acts as a strong temporal proxy in this dataset
- match_month ranks 2nd — may reflect scheduling or seasonal effects
- toss_winner_prior_win_rate is the top cricket-domain feature — prior form appears most relevant
- Red/blue dots in beeswarm show directional impact of high vs low feature values

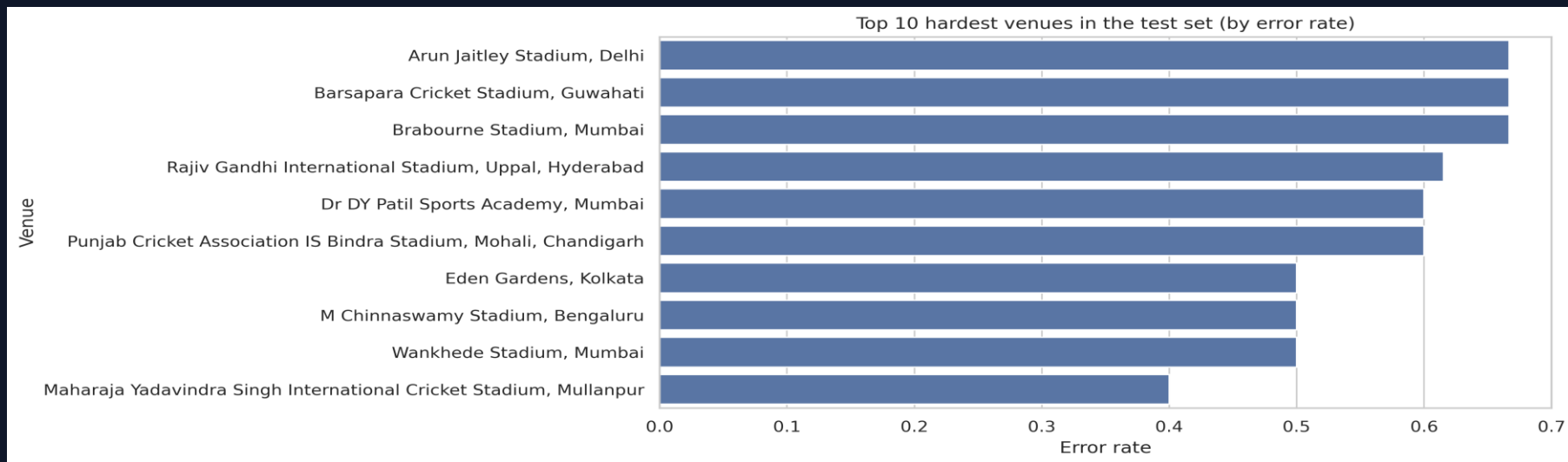
Model Explainability (SHAP)



Key Insights

Local: For this match, $f(x)=0.504$ (near coin-flip). Season year pushed prediction down (-0.01) while toss winner form pushed it up ($+0.01$) — features nearly cancel out, producing high uncertainty.

Error Analysis & Practical Insights



Error rate by venue — higher bars indicate harder-to-predict venues in this test split



Observations

- Delhi (Arun Jaitley Stadium) has the highest error rate (66.7% over 12 test matches) — appears harder for the model to predict in this split
- Venues with fewer test matches (Guwahati, Brabourne: 3 each) show high error rates but with low statistical confidence
- Model achieved highest accuracy at Dharamsala (75%, 4 matches) and Jaipur (70%, 10 matches)

Conclusion & Future Work



Project Summary

- Compared Random Search (50 trials) and Bayesian Optimization via Optuna (50 trials) for tuning a Keras MLP on IPL match prediction
- Both methods found fundamentally different architectures — optimization changes model behavior, not just accuracy
- No single model dominated all test metrics: the simple baseline led on ROC-AUC and log loss; tuned models shifted the precision-recall tradeoff
- It is a hard prediction task with limited features, tuning reveals tradeoffs rather than guaranteeing universal improvement

Future Work

- Richer pre-match features — player-level stats, recent form, weather, pitch reports
- Stronger tabular models — XGBoost, TabNet, FT-Transformer for comparison
- Better handling of class balance and probability calibration
- More domain-specific cricket features — head-to-head at specific venues, home advantage metrics
- Larger hyperparameter budgets and multi-objective optimization (accuracy + calibration jointly)